

Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: EJERCICIO 6

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Implementación y Código Fuente

El código fuente se estructuró para clasificar familias de cadenas en tres tipos de tokens (números, palabras y variables) utilizando cerraduras positivas y una variable de estado global.

- `int ContadorTokens=0;`: Variable global en C que persiste en memoria y contabiliza cuántos tokens válidos ha encontrado el autómata.
- `[0123456789]+`: Expresión regular que define un alfabeto numérico. El operador `+` obliga a que exista al menos un dígito, permitiendo repeticiones (ej. 1, 15, 999).
- `[a-zA-Z]+`: Expresión regular para secuencias compuestas exclusivamente por letras (mayúsculas o minúsculas).
- `[a-zA-Z][a-zA-Z0-9]*`: Regla para identificadores. Obliga a iniciar con una letra, y el operador `*` permite que le sigan cero o más letras o números.
- `yytext`: Variable interna tipo puntero de Lex que se invoca dentro de los `printf` para extraer e imprimir el valor exacto de la cadena que hizo coincidencia.

Tipos de entradas que recibe: Recibe secuencias alfanuméricas estructuradas, separadas por espacios o saltos de línea (ej. 1024, Variable1, texto).

Cadenas con las que el código fallaría (Errores Léxicos):

- Cadena `-15` o `+20`: El autómata no posee el signo en su alfabeto numérico. Al leer `"-15"`, imprimirá el `"-"` como basura y clasificará el `"15"` como NUMERO, rompiendo el valor lógico del entero.
- Cadena `var_temp`: El analizador no contempla guiones bajos. Leerá `var` como Identificador, expulsará `_` como eco (basura), y catalogará `temp` como Palabra, fragmentando una sola variable en múltiples pedazos.

Código fuente del archivo `seis.l`:

```
fmerino@fmerino: ~/COMPICD2026A/LAB-LEX-CarrascoA-M...
GNU nano 8.4 seis.l
%{
#include <stdio.h>
int ContadorTokens=0;
}%
%%
[0123456789]+      { printf("%d NUMERO \"%s\"\n", ++ContadorTok>
[a-zA-Z]+          { printf("%d PALABRA \"%s\"\n", ++ContadorTo>
[a-zA-Z][a-zA-Z0-9]* { printf("%d IDENTIFICADOR BASICO \"%s\"\n", >
%%
int main()
{
    yylex();
    return 0;
}
```

Entrada en entrada.txt:

```
fmerino@fmerino: ~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/SEIS
GNU nano 8.4 entrada.txt
1250
compilador
var123
99
x
```

2. Capturas de Ejecución

Se procesó un archivo de texto con múltiples sentencias que combinaban variables, asignaciones, números y los operadores a evaluar.

Ejecución final: Prueba de Procesamiento y Redirección (Consola y Archivo salida.txt):

```
fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/S
EIS$ lex seis.l
fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/S
EIS$ gcc lex.yy.c -lfl
fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/S
EIS$ ./a.out < entrada.txt
1 NUMERO "1250"

2 PALABRA "compilador"

3 IDENTIFICADOR BASICO "var123"

4 NUMERO "99"

5 PALABRA "x"

fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/S
EIS$ ./a.out < entrada.txt > salida.txt
fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/S
EIS$ cat salida.txt
1 NUMERO "1250"

2 PALABRA "compilador"

3 IDENTIFICADOR BASICO "var123"

4 NUMERO "99"

5 PALABRA "x"
```

Descripción de lo más relevante de lo observado:

- **Extracción de Lexemas:** El uso de la variable yytext demostró ser efectivo para capturar el contenido literal de los tokens (ej. extrayendo "1250" del flujo).
- **Priorización por longitud:** Al evaluar var123, el programa aplicó correctamente la regla de "IDENTIFICADOR BASICO" gracias a que la expresión regular abarcaba la longitud total de la cadena, en lugar de dividirla en letras y números.

3. Conclusión

El uso de operadores de repetición (+, *) en la teoría de expresiones regulares permite procesar componentes léxicos de longitud dinámica. Asimismo, el manejo del puntero yytext es indispensable para transferir el valor semántico de los tokens encontrados. No obstante, las reglas deben ser lo suficientemente robustas para evitar fragmentaciones ante caracteres especiales comunes.